

自然語言處理：詞嵌入與注意力

字元生成、情感分析與序列翻譯

書本第 14 章

建議 180 分鐘，含活動與休息

學習成果與課堂安排

- 0–50 分：字元語言模型與 embeddings。
- 60–110 分：tokenizer、情感分類與預訓練。
- 120–180 分：翻譯、beam search 與 attention。

成果：能對齊輸入與標籤，解釋 tokenizer、padding 與注意力的作用。

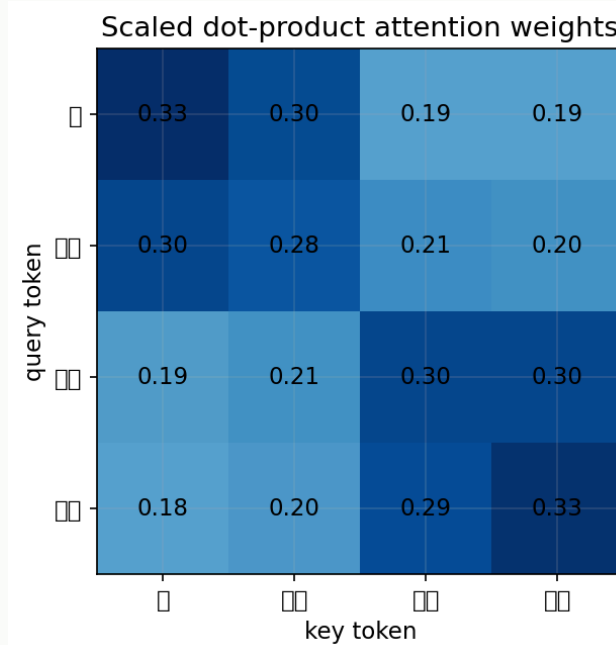
 **配套 Notebook**： `programs/notebooks/17_自然語言處理_詞嵌入與注意力.ipynb`。程式片段、實際圖表與表格可由此檔重現。

程式實驗與實際輸出

```
scores = Q @ K.T / sqrt(d)
weights = softmax(scores)
```

觀察：每列代表一個 query 對各 key 的權重，總和為 1；高權重不是因果證明。

參考程式：`programs/notebooks/17_自然語言處理_詞嵌入與注意力.ipynb`



字元語言模型的任務

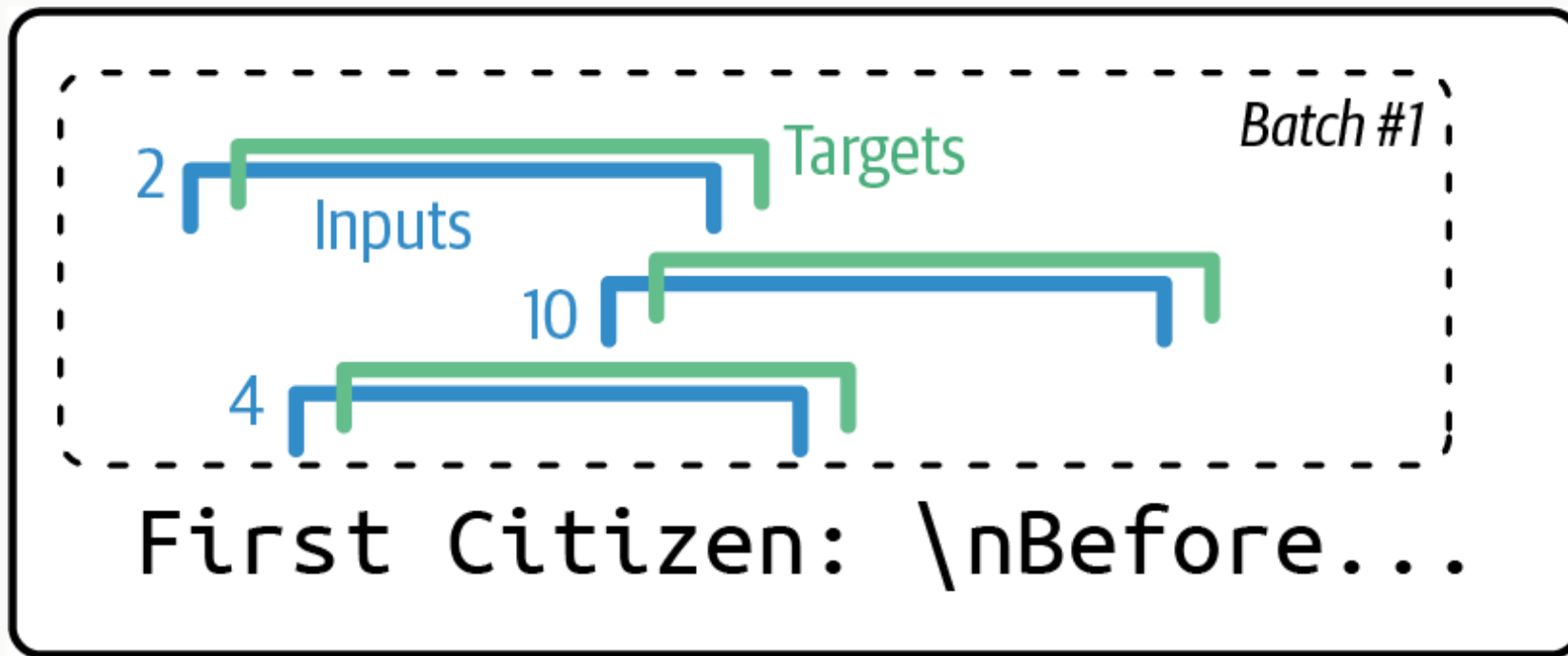
給定前面的字元，預測下一個字元的機率分布。

例：「機器學」後面可能接「習」。

訓練目標來自文字本身，不需逐字額外人工標註，但資料選擇仍影響結果。

生成時，把已產生的字元放回模型，逐步延長序列。

輸入與目標相差一個位置



每個輸入視窗對應向右位移一格的目標；最後一個字元也有下一步標籤。

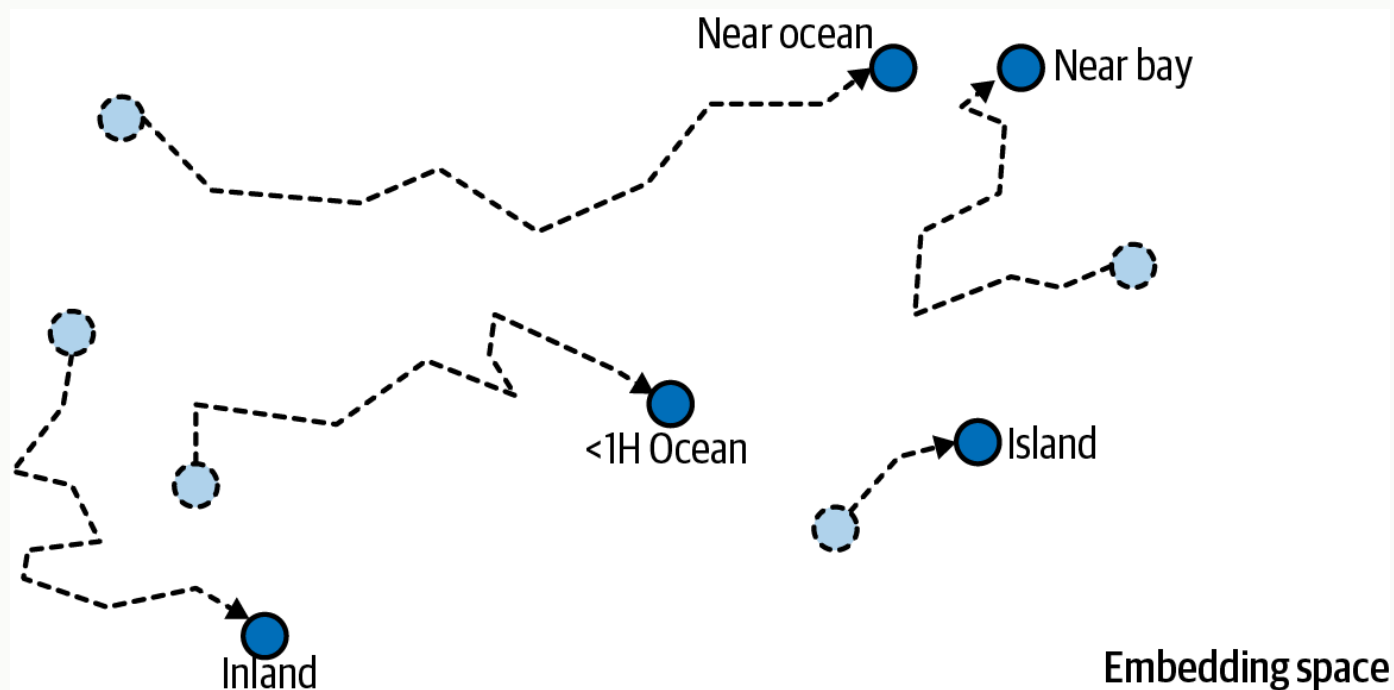
字元視窗手算

原始序列：A B C D E，視窗長度 4。

位置	1	2	3	4
輸入	A	B	C	D
目標	B	C	D	E

分類頭在每個位置輸出 vocab_size 個 logits。

Embedding 是可學習的查表



token ID 只用來查位置；數字 ID 的大小不代表語意遠近。

Embedding 的輸入與輸出

```
import torch
from torch import nn
embed = nn.Embedding(5, 3)
ids = torch.tensor([[3, 2], [0, 2]])
E = embed(ids)
print(E.shape) # 預期 [2, 2, 3]
```

五個 token，各自對應三維向量；重複 ID 會查到同一列參數。

作者程式：Cell 39 (14_nlp_with_rnn_and_attention.ipynb)

舊稿補充：One-hot、詞袋與詞向量

表示	保留什麼
單一 token 的 one-hot	token 身分
文件的 bag-of-words	詞頻，通常忽略順序
序列 embeddings	每個位置的稠密向量

Word2Vec 的 CBOW 與 Skip-gram 利用上下文學詞向量；本章也會端到端學 embeddings。

Char-RNN 的模型結構

```
# X: [B,T] 的 token IDs
embedding = nn.Embedding(vocab_size, 10)
gru = nn.GRU(10, 128, num_layers=2, batch_first=True)
head = nn.Linear(128, vocab_size)
outputs, states = gru(embedding(X))
logits = head(outputs).permute(0, 2, 1)
```

logits `[B,V,T]`，可搭配 `[B,T]` 目標使用 `CrossEntropyLoss`。

作者程式：Cell 42（結構節錄）（`14_nlp_with_rnnns_and_attention.ipynb`）

Temperature 與抽樣

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \quad T > 0$$

- 較小的 T 讓高分 token 更集中。
- 較大的 T 使分布更平坦。
- Greedy 每次選最大值，與隨機抽樣是不同策略。

溫度提高可能增加多樣性，也可能降低連貫性。

Stateful RNN 的資料順序

跨 batch 延續 hidden state 時，每個 batch 位置須接續同一段序列。

- 不任意 shuffle 連續批次。
- 狀態需適時 `detach()`，限制反向傳播範圍。
- 換文件或獨立序列時，重設狀態。

一般獨立視窗訓練則可打亂視窗，兩種做法不可混用。

情感分類的標籤單位

IMDB 範例對每篇評論預測正面或負面。

同一句出現「好」不必然代表正面，例如「好失望」。

先建立詞袋基準，再比較能利用順序的模型。

評估時檢查否定、反諷、長評論與領域轉移。

三種子詞 tokenizer

方法	核心想法	閱讀重點
BPE／byte-level BPE	反覆合併常見相鄰片段	位元組基底可處理更廣字元
WordPiece	依訓練準則建立子詞表	特殊token與子詞延續記號
Unigram	從候選子詞機率模型縮減詞表	可有不同切分可能

Tokenizer 必須搭配預訓練模型

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
batch = tokenizer(["A good movie", "Not good"],
                  padding=True, truncation=True,
                  max_length=128, return_tensors="pt")
print(batch["input_ids"].shape)
print(batch["attention_mask"])
```

英文模型僅作介面示範；中文任務須另選適合語言的模型與評估資料。

作者程式：Cell 125（改寫）（14_nlp_with_rnn_and_attention.ipynb）

Padding 與截斷

- Padding 讓同一 batch 的序列長度一致。
- Attention mask 標出真正 token 與補值位置。
- 截斷會丟掉文字，可能連否定或結論也一起刪掉。
- `padding_idx` 停止某列 embedding 更新，不等於 RNN 自動忽略 padding。

Packed sequence 避免讀入尾端補值

```
from torch.nn.utils.rnn import pack_padded_sequence
lengths = batch["attention_mask"].sum(dim=1)
E = embedding(batch["input_ids"])
packed = pack_padded_sequence(
    E, lengths.cpu(), batch_first=True, enforce_sorted=False)
outputs, h_n = gru(packed)
```

此用法假設有效 token 在前、右側 padding，且各長度大於 0。

作者程式：Cell 137（節錄）（`14_nlp_with_rnn_and_attention.ipynb`）

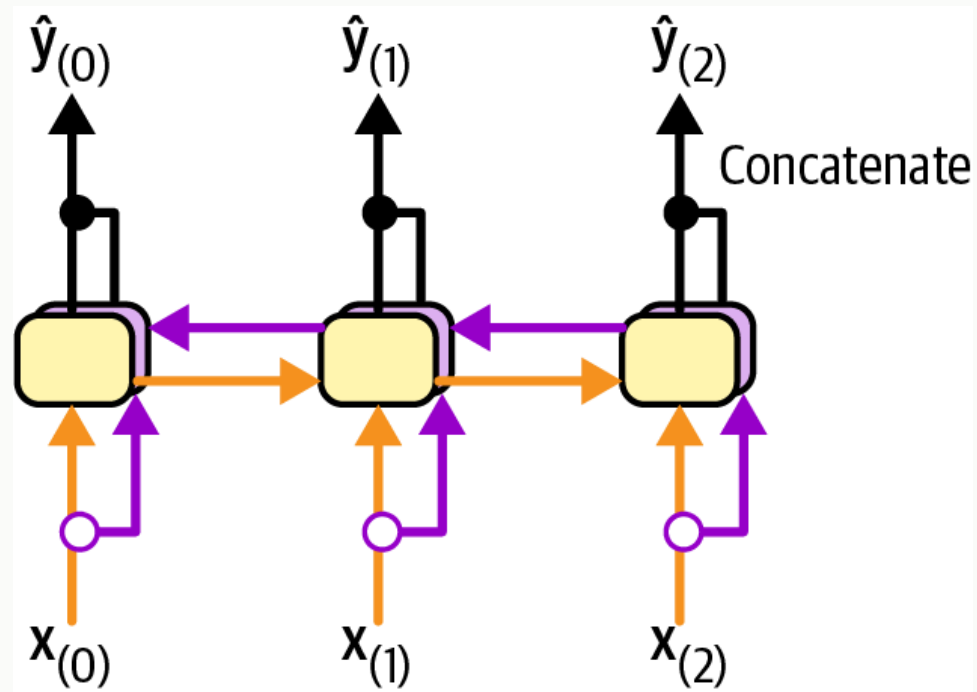
情感分類頭的損失與形狀

GRU 的句子表示接一個線性輸出，得到每句的正面 logit。

```
# logits 與 labels 都是 [B,1]
loss_fn = nn.BCEWithLogitsLoss()
loss = loss_fn(logits, labels.float())
proba_positive = logits.sigmoid()
```

訓練損失接原始 logits；只有要解讀機率時才做 Sigmoid。

雙向 RNN 的上下文



已知完整評論時可讀左右兩側；即時逐步預測未來時不能偷讀尚未到來的內容。

POS tagging 的逐 token 標籤

舊稿範例句： The quick brown fox jumps ◦

token	The	quick	brown	fox	jumps
Penn 標籤	DT	JJ	JJ	NN	VBZ

一個詞拆成多個子詞後，要約定標籤如何對齊與哪些位置忽略損失。

Stemming、lemmatization 與任務需求

- Stemming 以規則裁切字形，結果未必是字典詞。
- Lemmatization 常結合詞性還原詞元。
- 預訓練語言模型通常沿用原 tokenizer，不任意先做 stemming。

討論：若先刪除所有標點，情感或對話任務可能失去哪些訊號？

預訓練表示的兩個層次

表示	同一 token 在不同句子
靜態 embedding	通常查到同一向量
上下文表示	隨上下文改變

可凍結預訓練模型當特徵抽取器，也可微調部分或全部參數。

Task-specific class、Trainer 與 pipeline

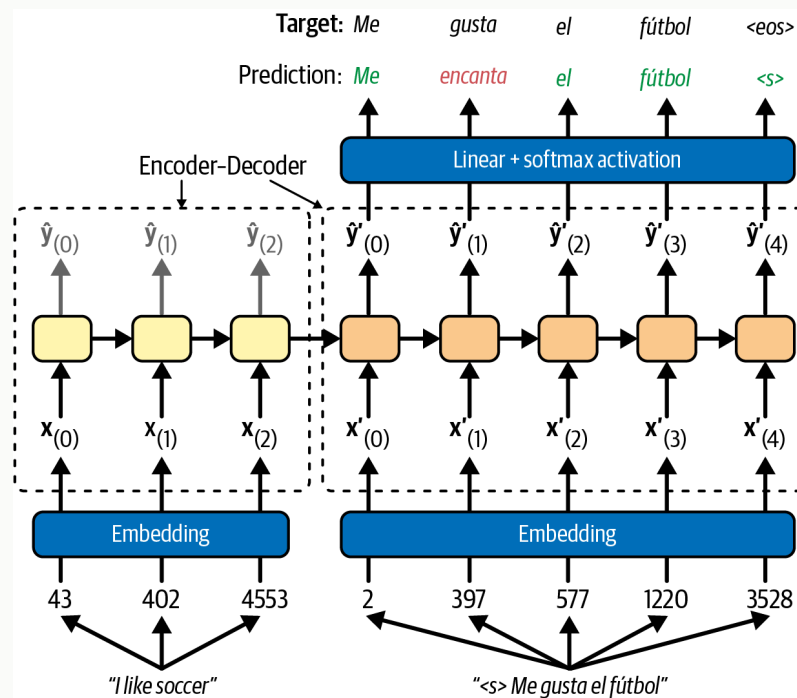
工具	幫忙處理什麼
AutoModelForSequenceClassification	預訓練主幹與分類頭
Trainer	訓練、評估、儲存與紀錄
pipeline	封裝前處理與推論

方便的介面仍需正確標籤、資料切分與評估指標。

偏差與公平性檢查

- 把人名、性別或地區詞替換後，比較預測是否不合理改變。
- 分群檢查錯誤率，不只看總準確率。
- 確認模型支援的語言與訓練領域。
- 將測例、結果與限制一起記錄，避免把分數當作人的評價。

Encoder-decoder 翻譯



Encoder 摘要來源句；decoder 依來源表示與已產生的目標文字預測下一詞。

Teacher forcing 的位移

目標句： <bos> I am a student <eos> 。

訓練角色	token 序列
Decoder 輸入	<bos> I am a student
監督標籤	I am a student <eos>

推論沒有標準答案可餵入，必須使用先前產生的 token 。

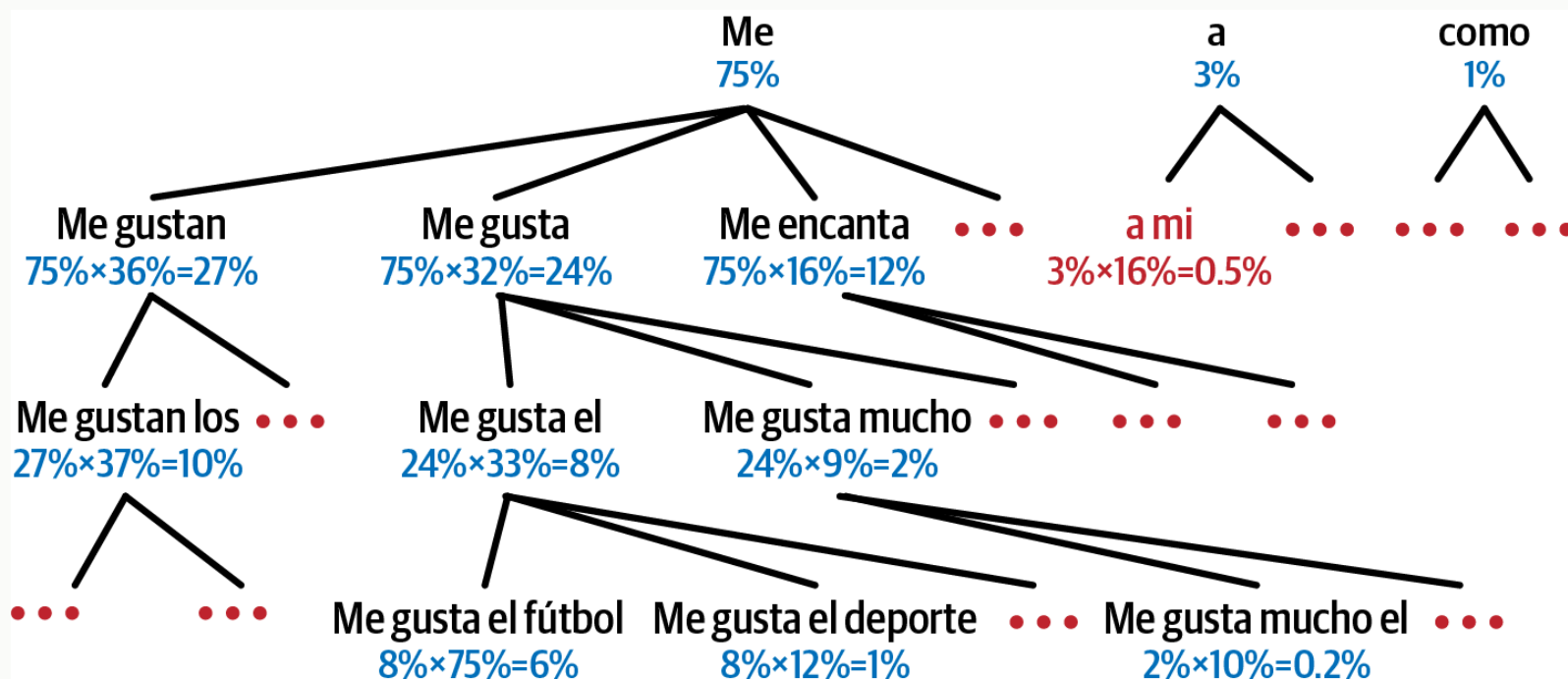
翻譯的固定向量瓶頸

長句的許多資訊若都壓在一個固定向量，可能難以保留。

先改進資料分桶、遮罩與解碼，再比較 attention。

翻譯評估可用自動指標輔助，仍需檢查漏譯、專有名詞與否定語意。

Beam search 保留多條候選



每步保留若干較高分的部分句子；寬度越大，計算與記憶體成本越高。

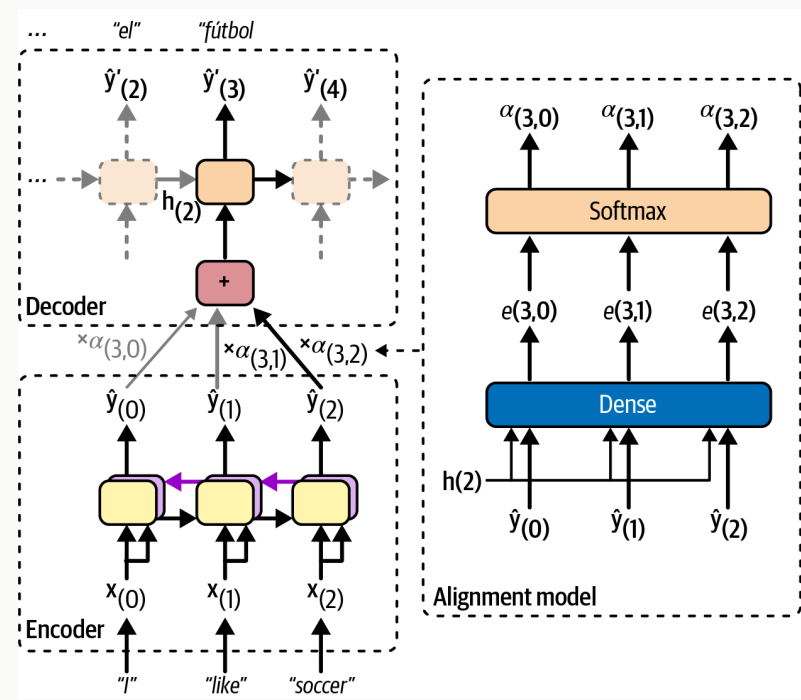
Greedy 不一定選到整句高機率

自編例：第一步 $A=0.6$ ， $B=0.4$ 。

- A 後面最佳續接的條件機率為 0.5，路徑機率 $0.6 \times 0.5 = 0.30$ 。
- B 後面最佳續接為 0.9，路徑機率 $0.4 \times 0.9 = 0.36$ 。

Greedy 先選 A，可能錯過 B 路徑；beam search 也不保證全域最佳。

Attention 動態讀取來源序列



每個輸出時間步對來源位置分配不同權重，再組合成當步 context。

注意力的三個步驟

$$e_{tj} = \text{score}(h_t, s_j), \quad \alpha_{tj} = \text{softmax}_j(e_{tj})$$

$$c_t = \sum_j \alpha_{tj} s_j$$

1. 比較目前 decoder 狀態與各來源狀態。
2. 沿來源位置做 Softmax。
3. 用權重加總來源 value。

注意力加權平均手算

若權重為 $[0.2, 0.3, 0.5]$ ，values 為 $[1, 0], [0, 2], [2, 2]$ ：

$$c = 0.2[1, 0] + 0.3[0, 2] + 0.5[2, 2] = [1.2, 1.6]$$

注意力權重是本次計算的分配；高權重不能單獨證明因果重要性。

程式導讀：Luong attention

作者 Notebook (`14_nlp_with_rnn_and_attention.ipynb`) : Cells 210–214。

- 找到 decoder outputs 與 encoder outputs 的矩陣乘法。
- 說明 Softmax 沿哪一軸計算。
- 檢查來源 padding 是否被遮罩。
- 追蹤 context 與 decoder 表示串接後的維度。

課堂活動：可重現的情感分類比較

20 分鐘：設計詞袋與 GRU 的比較表。

固定切分，記錄 tokenizer、長度、padding 與評估方式。

交付：兩種表示的資料形狀，以及包含否定、長句、領域差異的 6 個測例。

課後選讀：POS tagging 或英西翻譯，說明標籤對齊方式的差異。

離堂檢核

- token ID 的數字大小能表示語意距離嗎？
- `padding_idx` 是否讓 RNN 自動跳過補值？
- 訓練與推論的 decoder 輸入有何差異？
- Attention 為何不必只依賴最後一個 encoder state？

課後程式與延伸閱讀

- 作者第 14 章 Notebook (`14_nlp_with_rnn_and_attention.ipynb`) : 先執行 Setup , 再定位本課指定區段。
- 舊稿 `08_Recurrent Neural Networks.pptx` 、 `09_POS tagging.pptx` 、 `10_Attention.pptx` : 詞向量、序列標註與翻譯。

程式來源依教材核對版本標示；執行前確認資料、套件與運算資源。